



Apps Deployment on Cursor_@UMT



FAKULTI SAINS KOMPUTER DAN
MATEMATIK (FSKM), UNIVERSITI
MALAYSIA TERENGGANU (UMT)

Revision History

Revision Date	Previous Revision Date	Summary of Changes	Changes Marked
15/06/2021		First Issue	Fakhrul Adli Mohd Zaki
21/06/2021		“Upload & Retrieve File” tasks added	Fakhrul Adli Mohd Zaki
18/5/2026		- Update steps to reflect new server configuration - FAQ (BM) added	Fakhrul Adli Mohd Zaki

CONTENTS

TASK 1: Preparing JSP/Servlet Codes	1
TASK 2: Setting the Database parameters	3
TASK 3: Working with Cursor@UMT User Panel	9
TASK 4: Uploading file to Cursor@UMT server	21
TASK 5: Retrieving file from cursoR@umt Server	26
FAQ Penggunaan Server CURSOR_ (BAHASA MALAYSIA)	30

TASK 1: PREPARING JSP/SERVLET CODES

OBJECTIVE

To prepare JSP/Servlet Codes to avoid SQL Injection Attacks.

TASK DESCRIPTION

For this task, the student will manually scan their codes to make sure that their project is using JSP PreparedStatement for querying the database.

ESTIMATED TIME

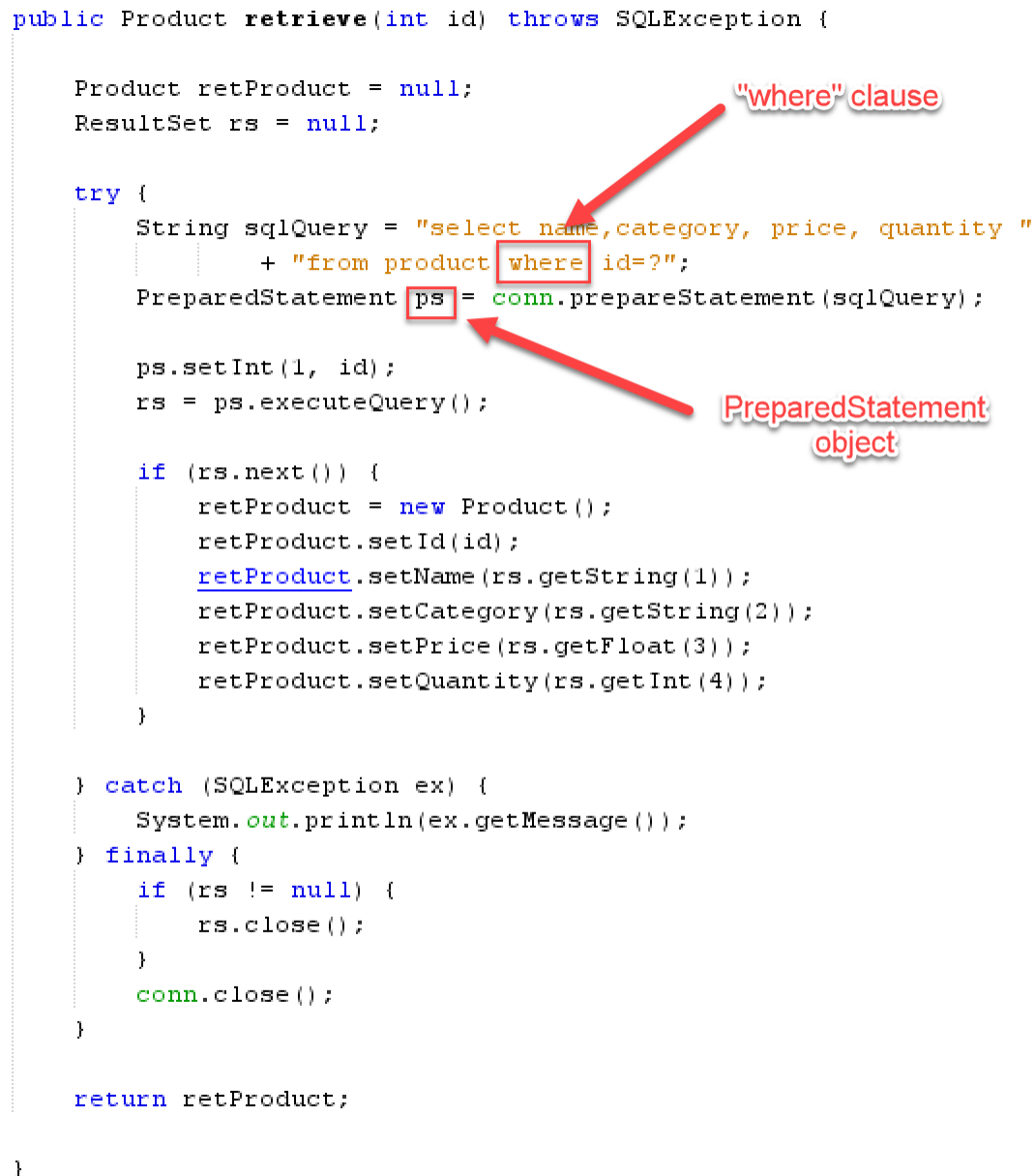
10 Minutes

STEPS:

1. Open your JSP project from Netbeans (or any IDEs).

2. Go to the lines where SQL queries have been inserted. Make sure all the SQL query lines with **"where"** clause is executed in a PreparedStatement object. As example,

```
public Product retrieve(int id) throws SQLException {  
  
    Product retProduct = null;  
    ResultSet rs = null;  
  
    try {  
        String sqlQuery = "select name,category, price, quantity "  
            + "from product where id=?";  
        PreparedStatement ps = conn.prepareStatement(sqlQuery);  
  
        ps.setInt(1, id);  
        rs = ps.executeQuery();  
  
        if (rs.next()) {  
            retProduct = new Product();  
            retProduct.setId(id);  
            retProduct.setName(rs.getString(1));  
            retProduct.setCategory(rs.getString(2));  
            retProduct.setPrice(rs.getFloat(3));  
            retProduct.setQuantity(rs.getInt(4));  
        }  
  
    } catch (SQLException ex) {  
        System.out.println(ex.getMessage());  
    } finally {  
        if (rs != null) {  
            rs.close();  
        }  
        conn.close();  
    }  
  
    return retProduct;  
}
```



The diagram shows two red arrows pointing to specific parts of the code. One arrow points from the text **"where" clause** to the `where id=?` part of the SQL query string. The other arrow points from the text **PreparedStatement object** to the `ps` variable in the `PreparedStatement ps = conn.prepareStatement(sqlQuery);` line.

3. If all the codes have been fixed, ready for Task 2.

TASK 2: SETTING THE DATABASE PARAMETERS

OBJECTIVE

To configure the required parameters for database connection in JSP/Servlet.

TASK DESCRIPTION

During this task, the student will update the parameters for database connection. There are three important parameters:

- Database name
- Username
- Password

ESTIMATED TIME

10 Minutes

STEPS:

1. Open the file for the database connection file of your web application. See the following screenshot as an example:

```
/**
 *
 * @author Fakhru Adli
 */
public class ConnectionManager {

    private static final String URL = "jdbc:mysql://localhost:3306/onlinestore";
    private static final String DRIVENAME = "com.mysql.jdbc.Driver";
    private static final String USERNAME = "root";
    private static final String PASSWORD = "";
    private static Connection conn;

    public static Connection getConnection() {
        try {
            Class.forName(DRIVENAME);
            try {
                conn = DriverManager.getConnection(URL, USERNAME, PASSWORD);
            }
            catch (SQLException ex) {
                System.out.println(ex.getMessage());
            }
        }
        catch (ClassNotFoundException ex) {
            System.out.println(ex.getMessage() + " not found!");
        }
        return conn;
    }
}
```

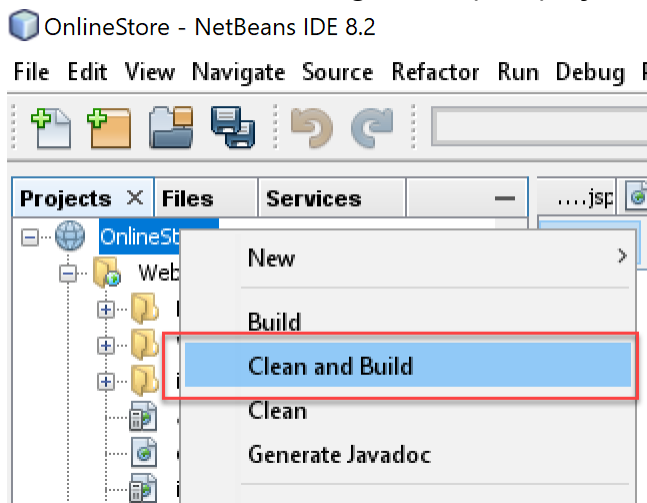
- Now, change the parameters according to the username and password which are given by the Cursor_@UMT administrator. For example, the username gives for this user is **s12345** and password **sw2\$f3**. **Note:** Use small letter "s". Cursor_ is running on Linux and it always the best practice to use small letters for database name, table and field name.

Parameter	Parameter at local	Parameter for hosting
Database Name	online	s12345_onlinestore
Username	root	s12345
Password	[any given password]	sw2\$f3

- So, the final configuration in the codes for the database will be as follows:

```
private static final String URL = "jdbc:mysql://localhost:3306/s12345_onlinestore";
private static final String DRIVERNAME = "com.mysql.jdbc.Driver";
private static final String USERNAME = "s12345";
private static final String PASSWORD = "sw2$f3";
```

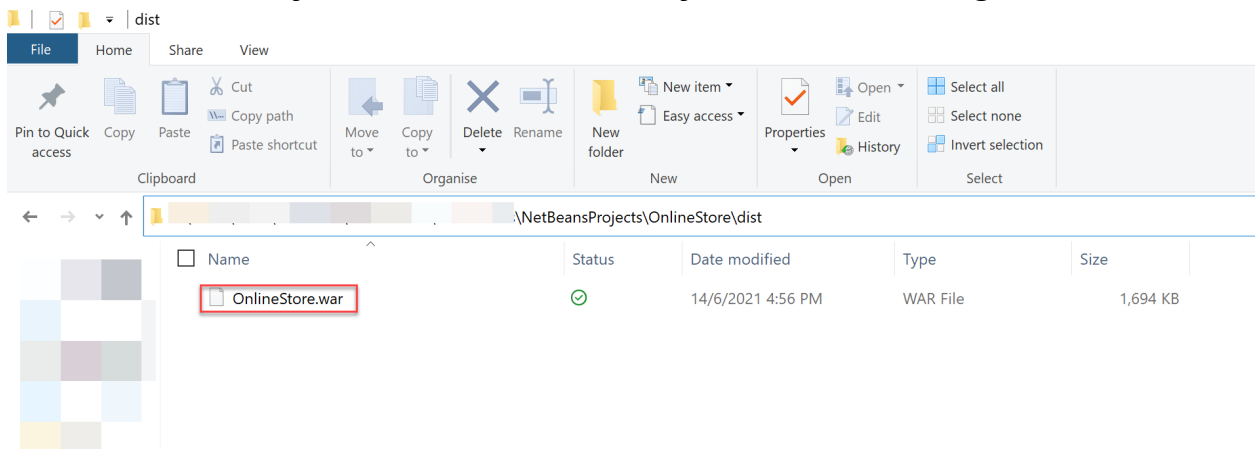
- Bear in mind that the letters are case-sensitive. For e.g, s12345 is different from S1245.
- After done with the configuration, we are now ready to **Clean & Build** our web application.
- Go to the Netbeans IDE, right-click your project name, then select **Clean & Build**.



7. Wait for a few seconds for the process to finish. A successful build will show the output below at Netbeans:

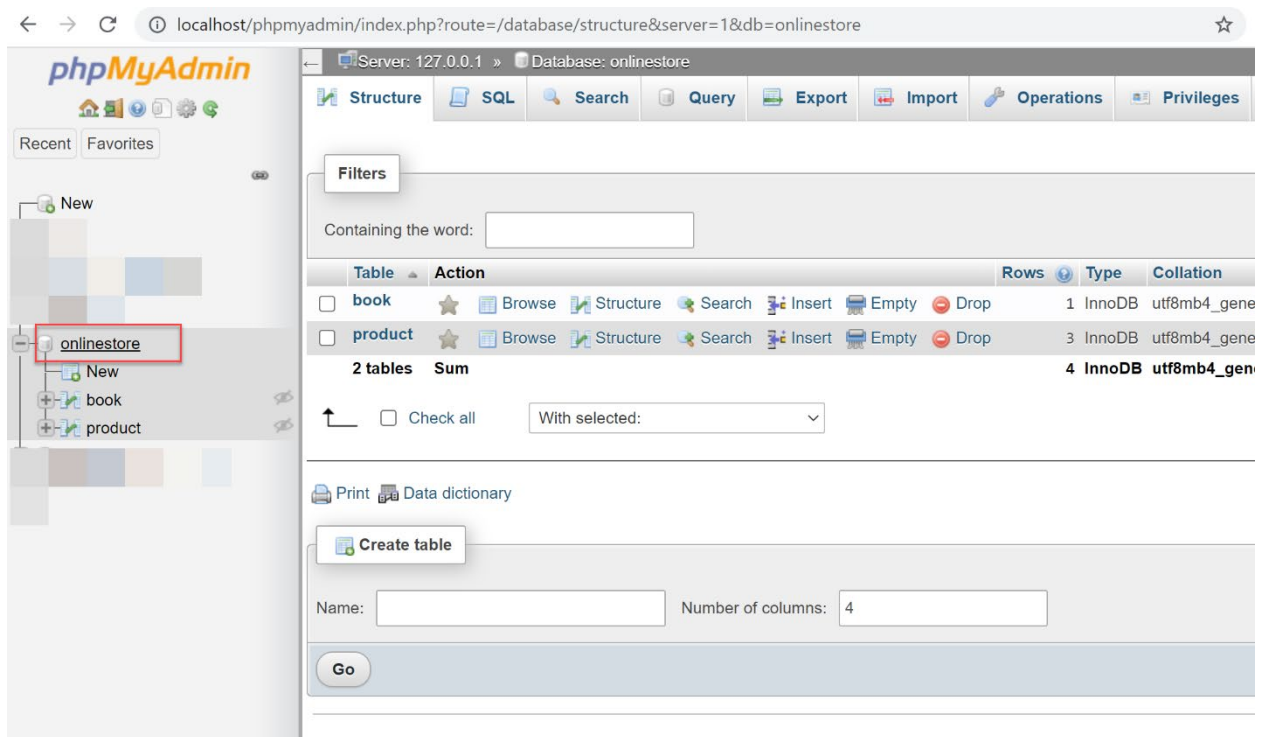
```
Output - OnlineStore (clean,dist)
Copying 1 file to NetBeansProj
Copying 1 file to NetBeansProj
Copying 1 file to NetBeansProj
library-inclusion-in-manifest:
Created dir:
Created dir:
Compiling 6 :
compile:
compile-jsp:
Created dir:
Building jar:
do-dist:
dist:
BUILD SUCCESSFUL (total time: 7 seconds)
```

8. The result of this step is a deployment file known as the WAR file. We can find the file at [Your Project Location]\[Project Name]\dist folder. In this example, the WAR file for this project is located at [Project Location]\ NetBeansProjects\OnlineStore\dist. **Note:** This depend on what build automation tools you are using. If you are using Maven, the WAR file can be located at [Project Location]\ NetBeansProjects\OnlineStore\target.

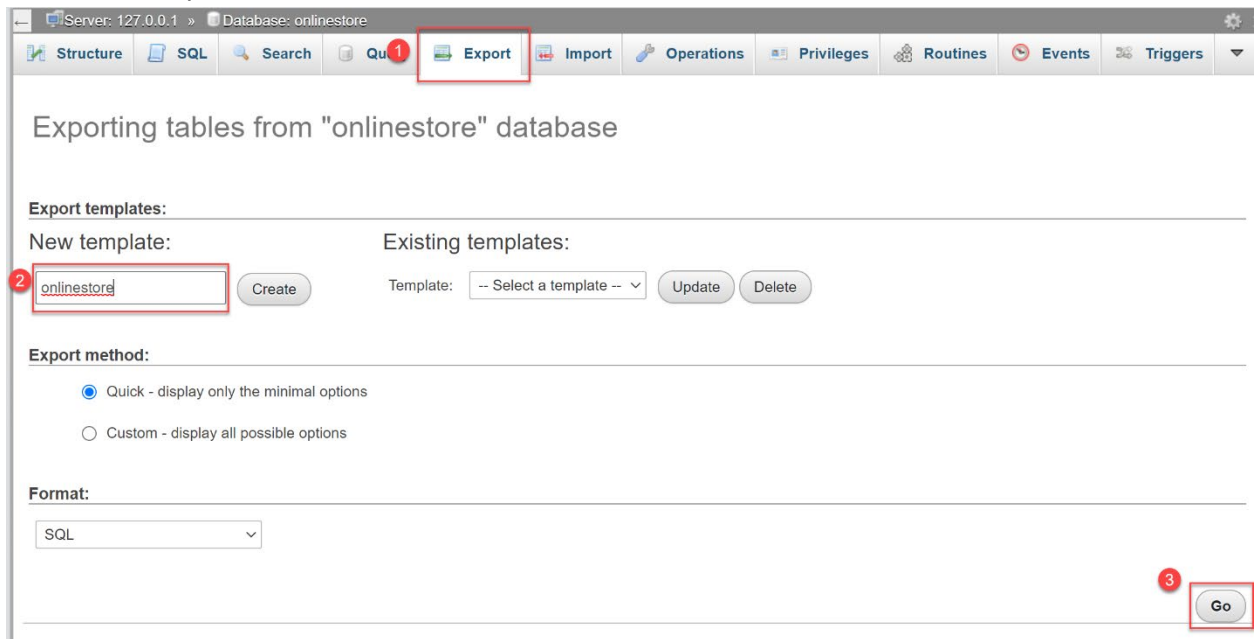


9. Rename the WAR file with your matric number and keep the WAR extension. As example OnlineStore.ware should be renamed to s12345.war. **This is very important step, otherwise Cursor_ will ignore your WAR file.**
10. Take note of the location of the WAR file. We are going to upload the file for the incoming task.

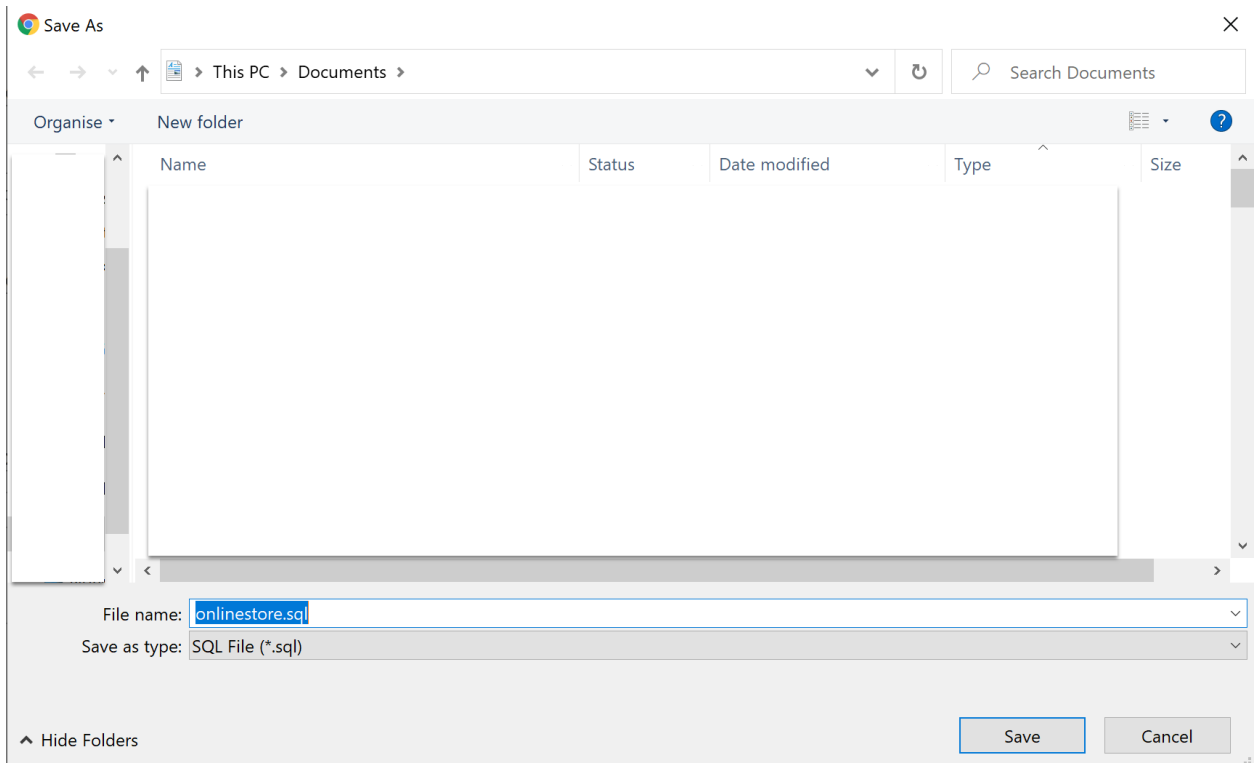
11. At this stage, we have finished the configuration for JSP/Servlet part. Let's move to export the database.
12. Open the web browser, then go to the phpMyAdmin page. Make sure you start the Apache and MySQL server first before doing this (it can be done at XAMPP Control Panel).
13. Click on the link to the database schema.



14. Then, click Export, fill in the name of the database and click **Go**.



15. Save the SQL file at a location that is easy to find. We need the file later.



16. The above step is the last step at the local machine. Two important files are ready to be uploaded to the server:

- a. WAR file: for project deployment
 - b. SQL file: for database export.
17. Let's continue to Task 3, which will involve the remote server (cursor_@UMT).

TASK 3: WORKING WITH CURSOR@UMT USER PANEL

OBJECTIVE

To login to the CURSOR_@UMT user panel and continue with application deployment and database settings.

TASK DESCRIPTION

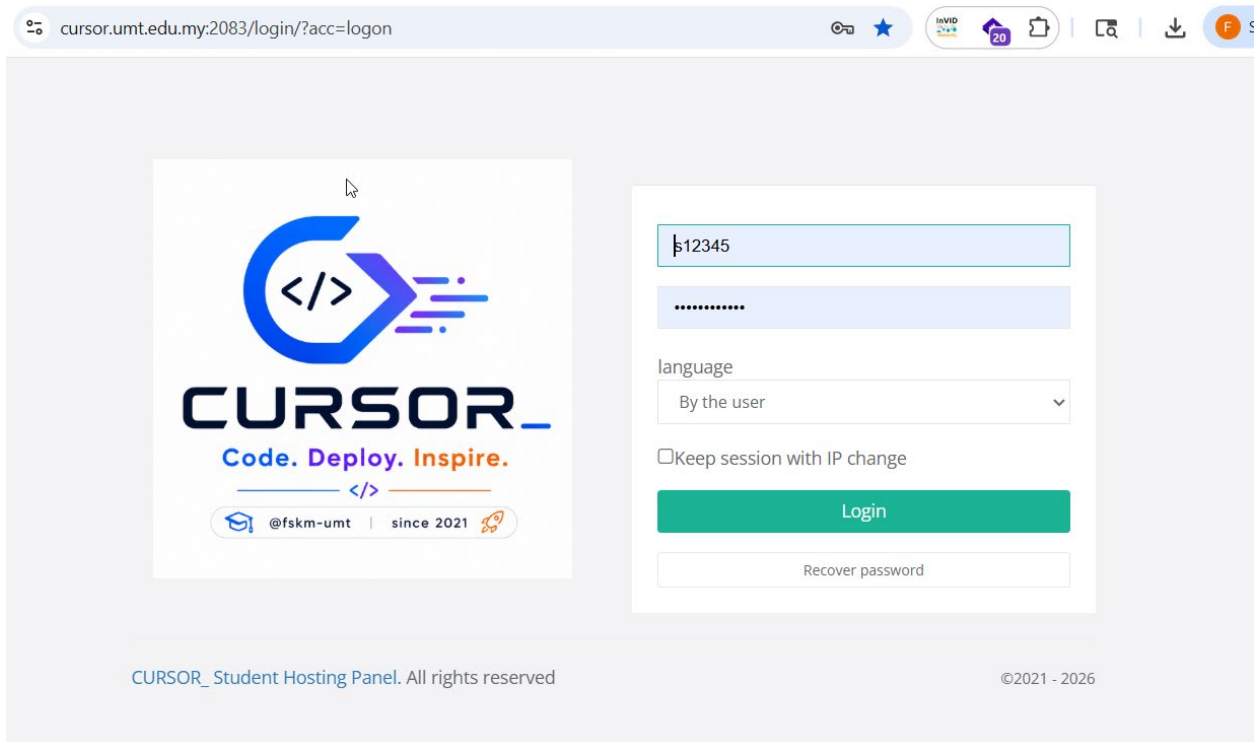
For this task, the student will log in to the user panel to upload the WAR file and configure the database.

ESTIMATED TIME

60 Minutes

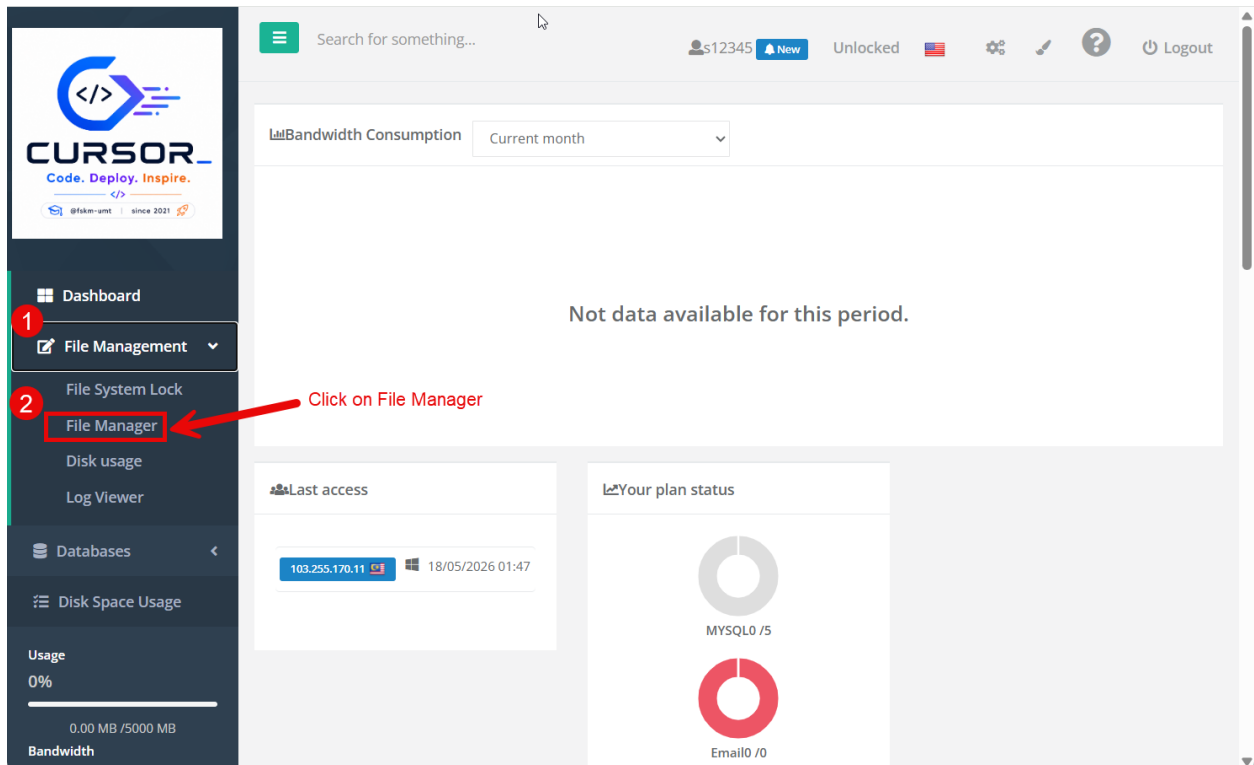
STEPS:

1. Open a web browser, then go to the administration URL provided by the system administrator <https://cursor.umt.edu.my:2083/login/?acc=logon>.
2. Log in to the user panel using the username and password provided by the administrator.



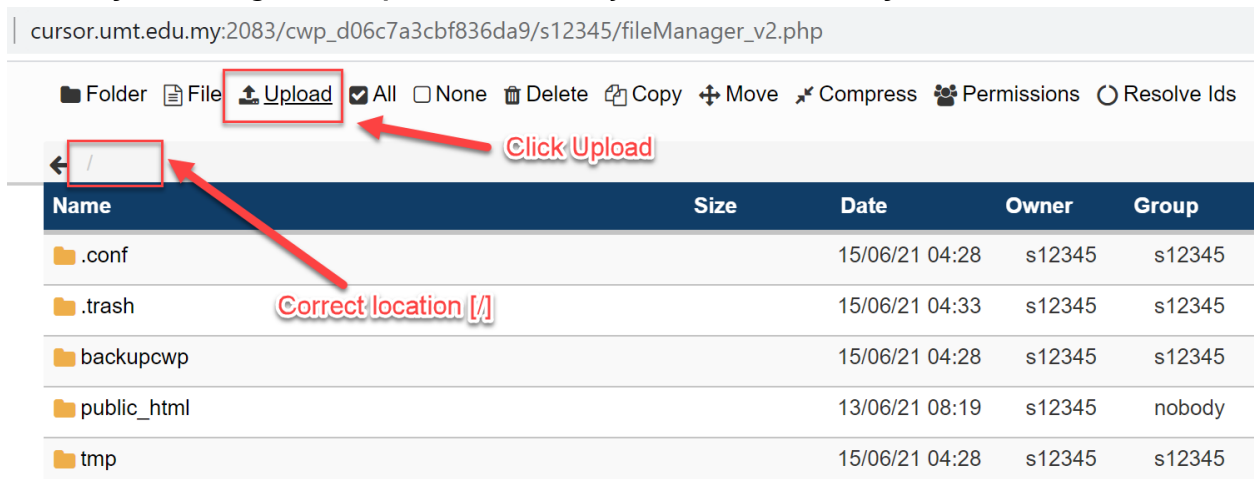
The screenshot displays the login interface for the CURSOR_ Student Hosting Panel. On the left, there is a logo featuring a stylized blue 'C' with a code symbol '</>' inside, followed by the text 'CURSOR_ Code. Deploy. Inspire.' and a footer with '@fskm-umt | since 2021'. On the right, the login form includes a username field (containing '12345'), a password field (masked with dots), a language dropdown menu (set to 'By the user'), a checkbox for 'Keep session with IP change', a green 'Login' button, and a 'Recover password' link. The browser's address bar shows the URL 'cursor.umt.edu.my:2083/login/?acc=logon'. The footer of the page reads 'CURSOR_ Student Hosting Panel. All rights reserved' and '©2021 - 2026'.

3. After successful login, you will see a dashboard that shows some important information such as the bandwidth consumption and IP Address of Last Access. Now, let's begin the web application deployment process by choosing **File Management** from the left menu and click on the **File Manager** link.

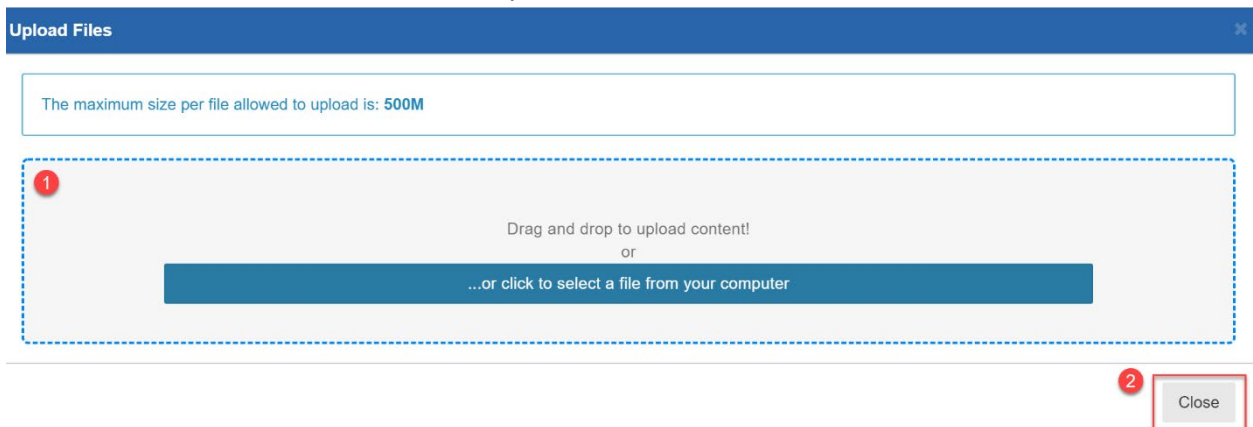


4. Next, click on **Upload**.

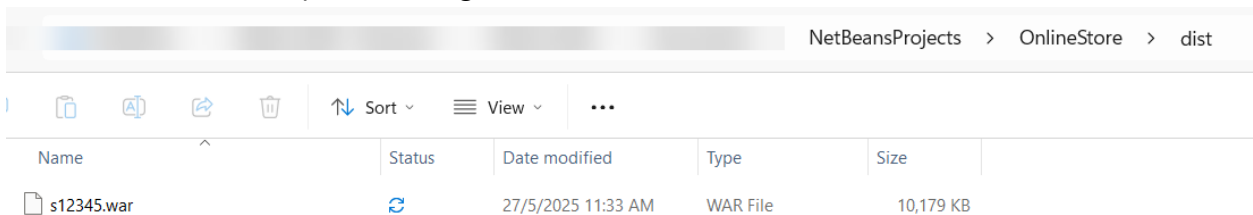
*Note: Before clicking on the **Upload**, make sure you are at the correct folder which is in /.*



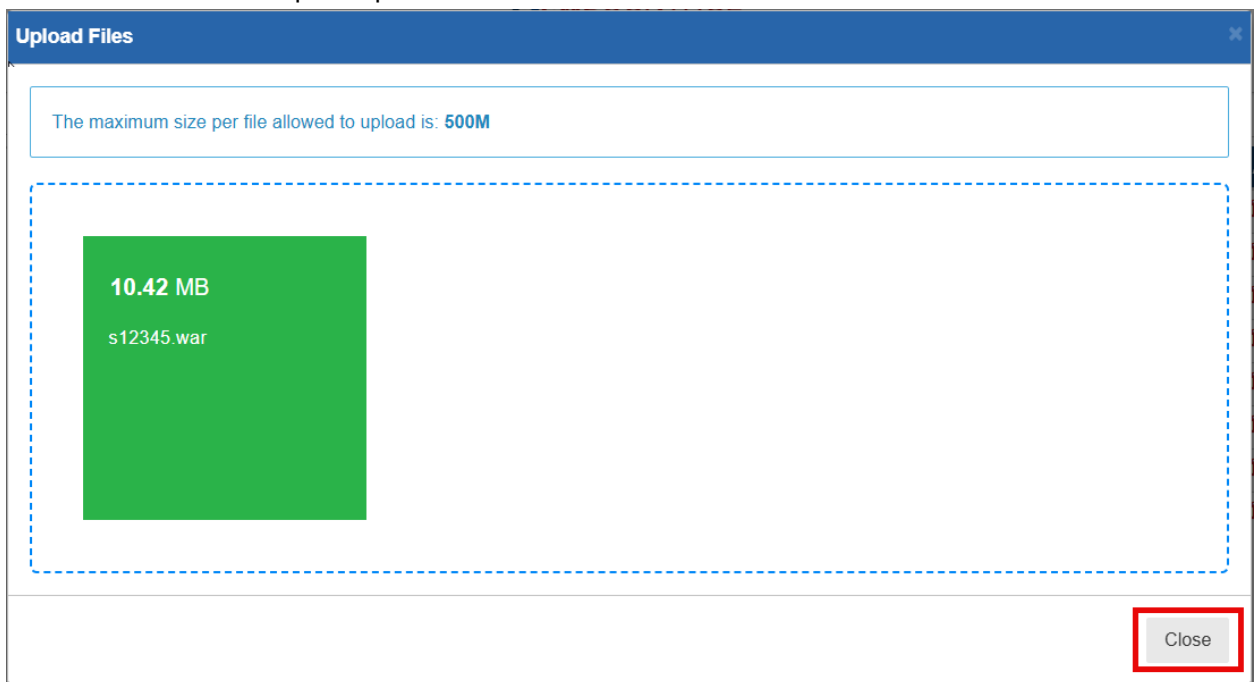
5. A dialogue box as below will appear. Select the WAR file by dragging the file to the box or click on the button to select it manually.



6. This dialogue box appears if you click on **...or click to select a file from your computer**. Select the WAR file that has been generated during **Task 2: Step 6 & 7**. **Note:** As mentioned in Task 2, by default the location of the WAR might be different, depending on the build automation tools that you are using; Gradle or Maven.



- Click **Close** after the upload process has finished.



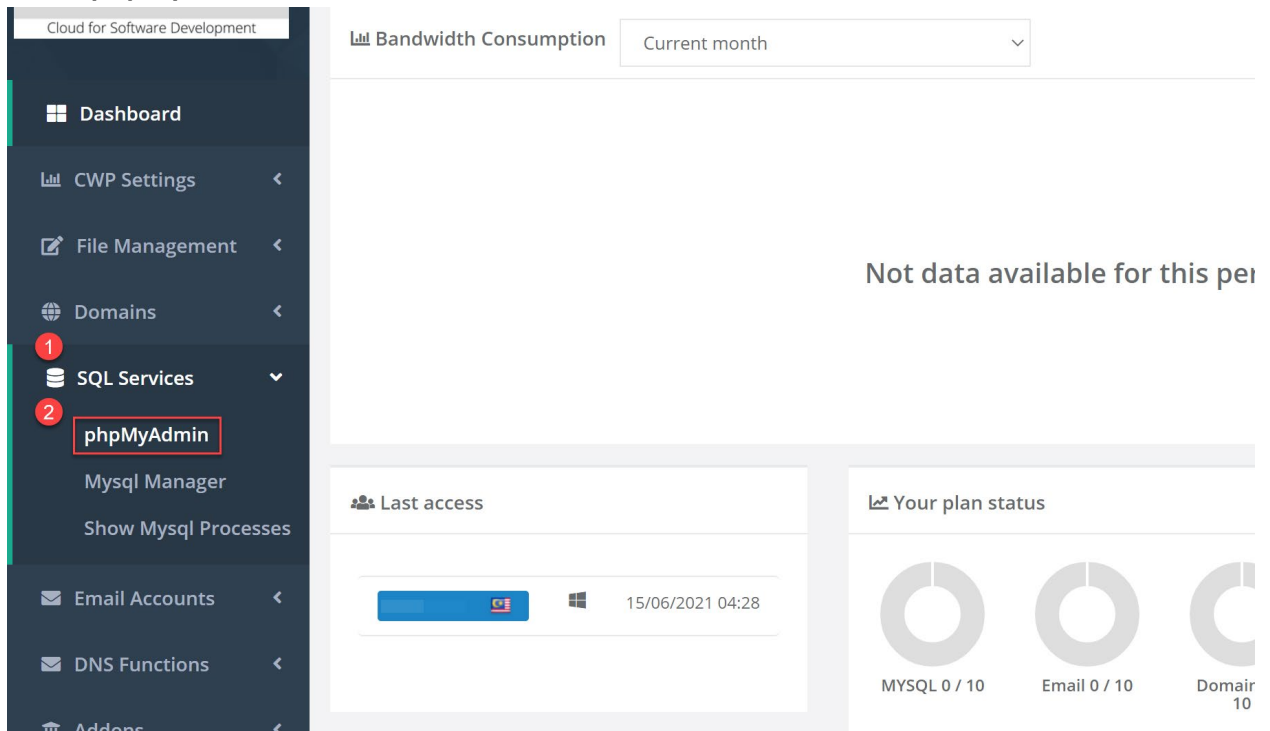
- Confirm that the WAR file upload is successful by looking at the list of files in /.

Folder File Upload ☒ All ☐ None Delete Copy Move Compress Permissions Resolve Ids

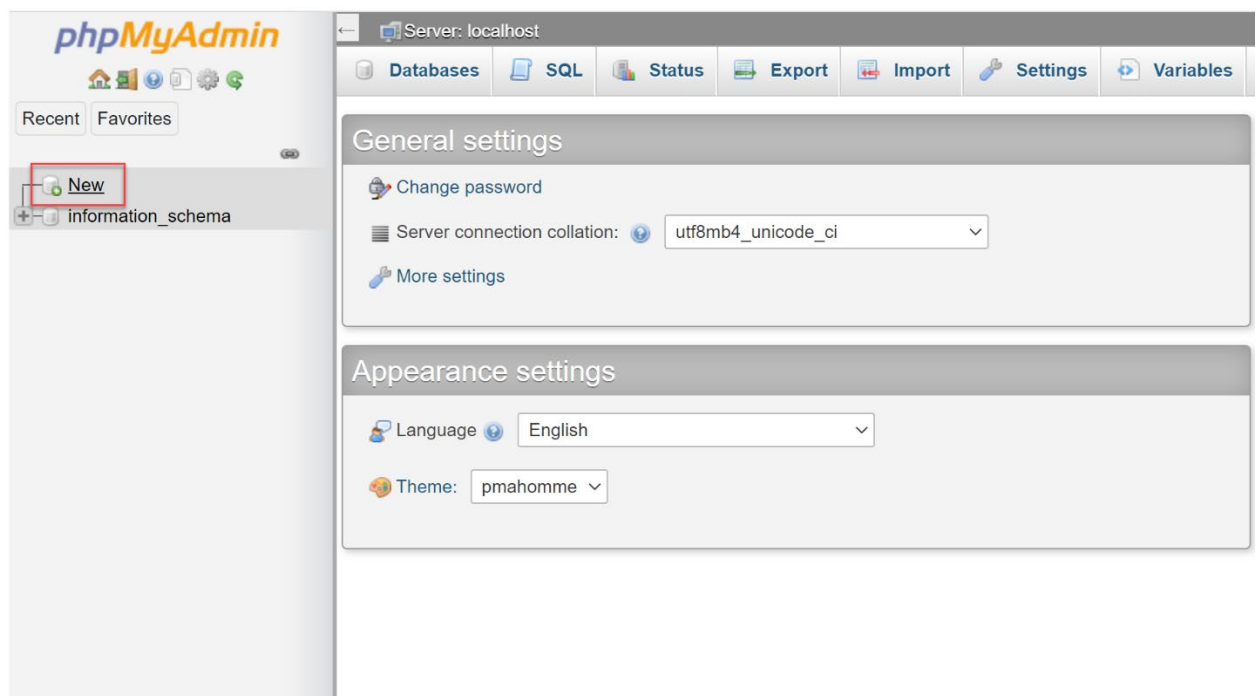
All Your Files

	Name	Date	Owner	Group	Permissions	Actions
+	.conf	18/05/26 01:47	s12345	s12345	755	
+	.trash	18/05/26 02:18	s12345	s12345	755	
+	backupcpwp	18/05/26 01:47	s12345	s12345	755	
+	public_html	24/01/17 20:59	s12345	nobody	750	
+	tmp	18/05/26 01:47	s12345	s12345	755	
	.bash_logout	30/04/24 14:14	s12345	s12345	644	
	.bash_profile	30/04/24 14:14	s12345	s12345	644	
	.bashrc	30/04/24 14:14	s12345	s12345	644	
	s12345.war	18/05/26 02:19	s12345	s12345	644	

9. Now, let's move on to the database setup. Click on **SQL Services** on the left menu and select **phpMyAdmin**.



10. You will see the phpMyAdmin homepage as below. Then, click **New** on the left menu to create a new database schema.



11. Give the name of the database schema similar to the one in the local host except this one should be written after the word **[username]_**. For example, **s12345_onlinestore** shown in the screenshot below is a valid name. Click **Create** when it is confirmed that everything is correct.

Server: localhost

Databases SQL Export Settings Variables

Databases

1 Create database

s12345_onlinestore latin1_swedish_ci Create

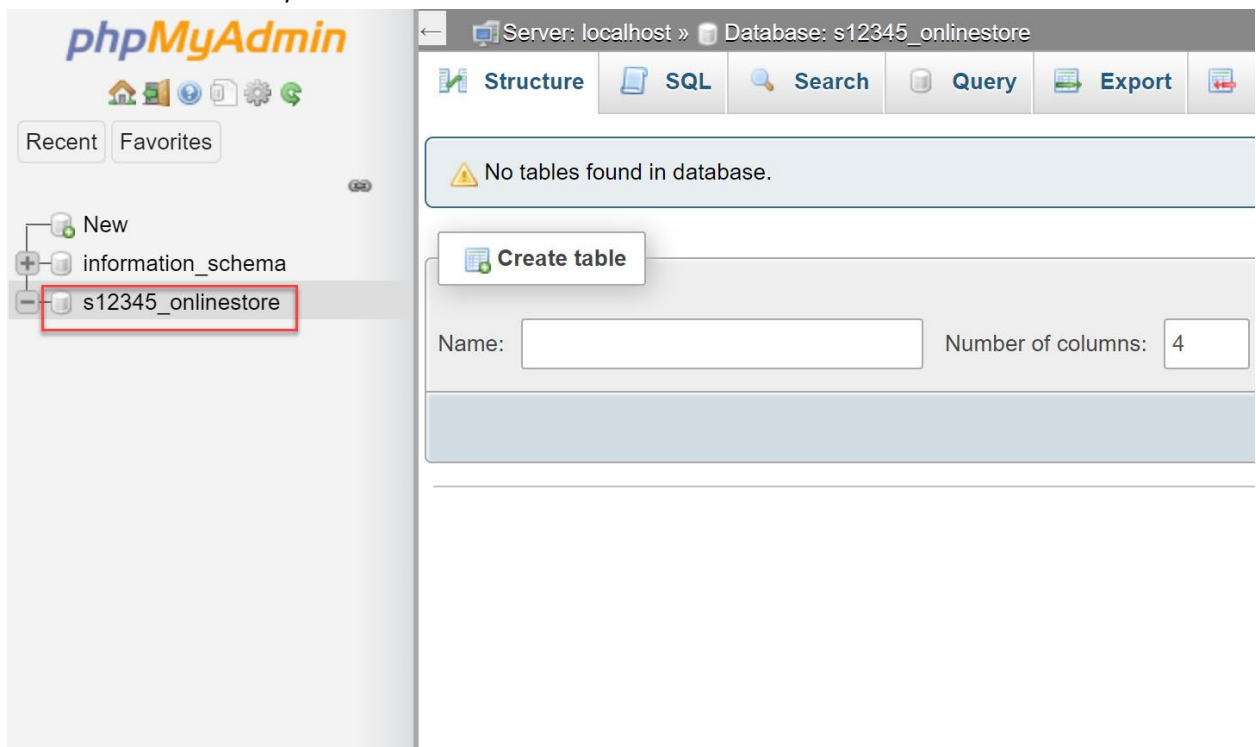
Database	Collation	Action
information_schema	utf8_general_ci	Check privileges

Total: 1

⚠ Note: Enabling the database statistics here might cause heavy traffic between the web server and the database server.

- [Enable statistics](#)

12. You will see the newly created database schema has been listed on the left menu.



13. Next, click **Import** on the top menu and click on **Choose File**. Choose the SQL file that has been created during **Task 2: Step 10 – Step 13**.



Importing into the database "s12345_onlinestore"

File to import:

File may be compressed (gzip, bzip2, zip) or uncompressed.

A compressed file's name must end in **.[format].[compression]**. Example: **.sql.zip**

Browse your computer: **Choose File** No file chosen (Max: 500MiB)

You may also drag and drop a file on any page.

Character set of the file: utf-8

Partial import:



Allow the interruption of an import in case the script detects it is close to the PHP timeout limit. *(This might be a good way to im*

Skip this number of queries (for SQL) starting from the first one: 0

14. Scroll down the page and click **Go** to proceed.

Partial import:



Allow the interruption of an import in case the script detects it is close to the PHP timeout limit. *(This might be a good way to import large files, however it can break transactions.)*

Skip this number of queries (for SQL) starting from the first one: 0

Other options:

☒ Enable foreign key checks

Format:

SQL

Format-specific options:

SQL compatibility mode:

NONE

☒ Do not use AUTO_INCREMENT for zero values

Go

15. You will see a confirmation message of a successful import and a list of tables can be seen on the left menu.

The screenshot displays the phpMyAdmin web interface. On the left sidebar, the database 's12345_onlinestore' is selected, and its tables 'book' and 'product' are listed. The main panel shows a confirmation message: 'Import has been successfully finished, 17 queries executed. (onlinestoreV2.sql)'. Below this, several status messages indicate that MySQL returned an empty result set for various queries. The SQL dump content is visible, including comments about phpMyAdmin version, host, and PHP version, as well as SQL mode settings.

Server: localhost » Database: s12345_onlinestore

Structure SQL Search Query Export Import Operations

Import has been successfully finished, 17 queries executed. (onlinestoreV2.sql)

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0001 seconds.)

```
-- phpMyAdmin SQL Dump -- version 5.0.4 -- https://www.phpmyadmin.net/ -- -- Host: 127.0.0.1
version: 10.4.17-MariaDB -- PHP Version: 8.0.1 SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO"
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0000 seconds.)

```
START TRANSACTION
```

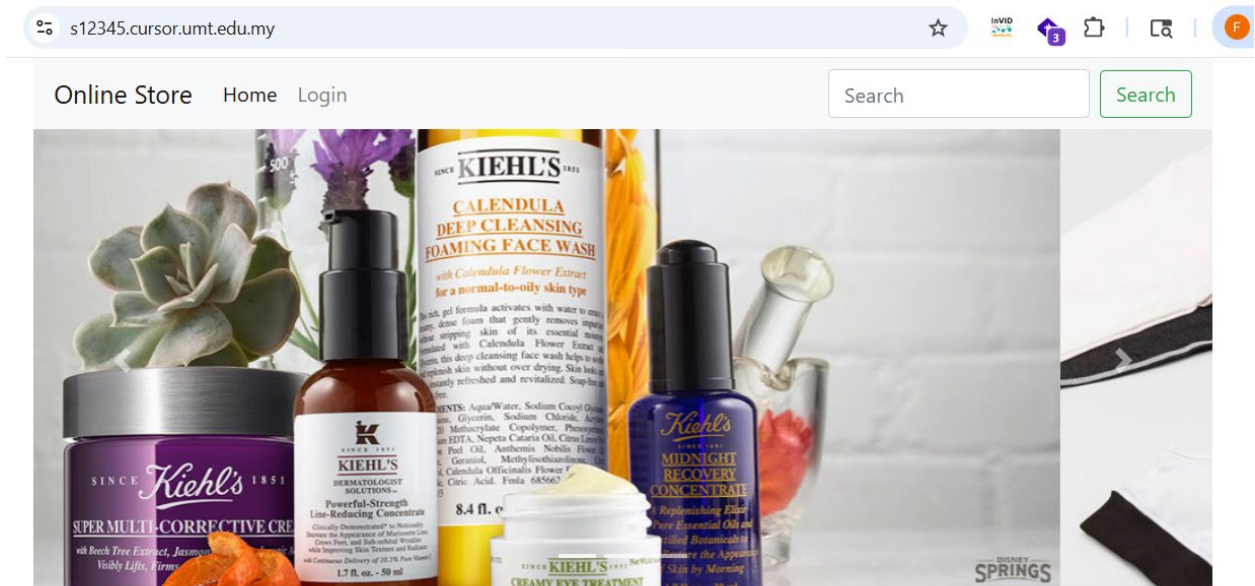
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0001 seconds.)

```
SET time_zone = "+00:00"
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0001 seconds.)

```
/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */
```

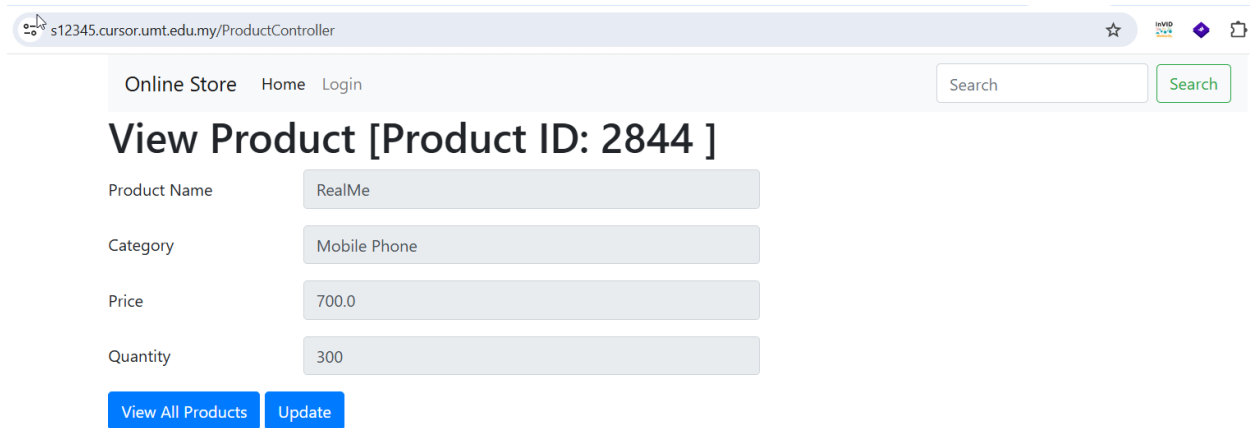
16. Well done! Now we have successfully set up our web application at the Cursor@UMT server. Let's test the apps by going to the URL **https://[your matrix number].cursor.umt.edu.my**. As seen in the below example, the home page of the OnlineStore web application. **https://s12345.cursor.umt.edu.my**



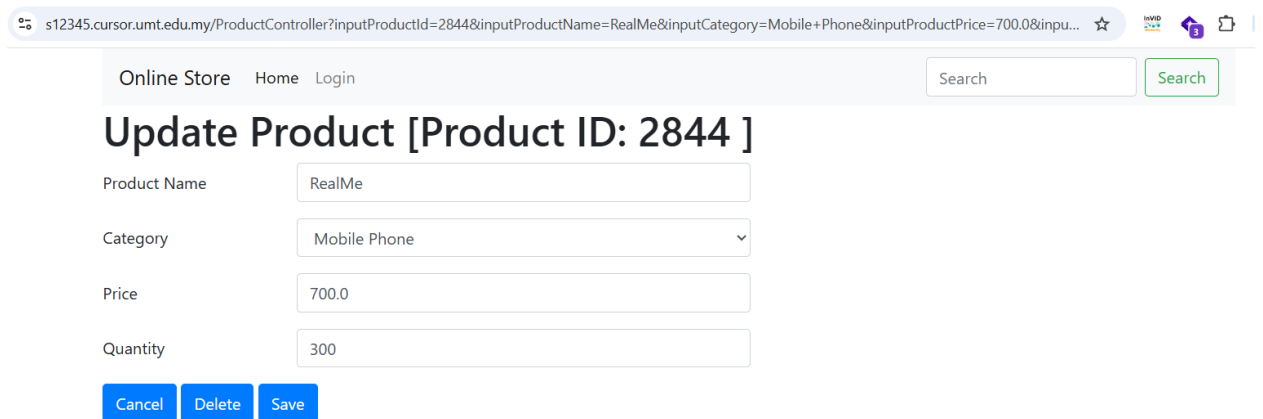
17. It always a good practice to test all the CRUD operations of your web application. Maybe we can start with creating a new record.

A screenshot of the 'Add New Product' form in the online store. The browser's address bar shows the URL 's12345.cursor.umt.edu.my/addNewProduct.jsp'. The page has a navigation bar with links for 'Online Store', 'Home', and 'Login', followed by a search bar with a 'Search' button. The main content area has a heading 'Add New Product [Product ID:2844]'. Below the heading are four input fields: 'Product Name' with the value 'RealMe', 'Category' with a dropdown menu showing 'Mobile Phone', 'Price' with the value '700', and 'Quantity' with the value '300'. At the bottom of the form are two buttons: 'Reset' and 'Save'.

18. After successful saved (C) and retrieve (R), we can proceed to update(U) or delete (D).

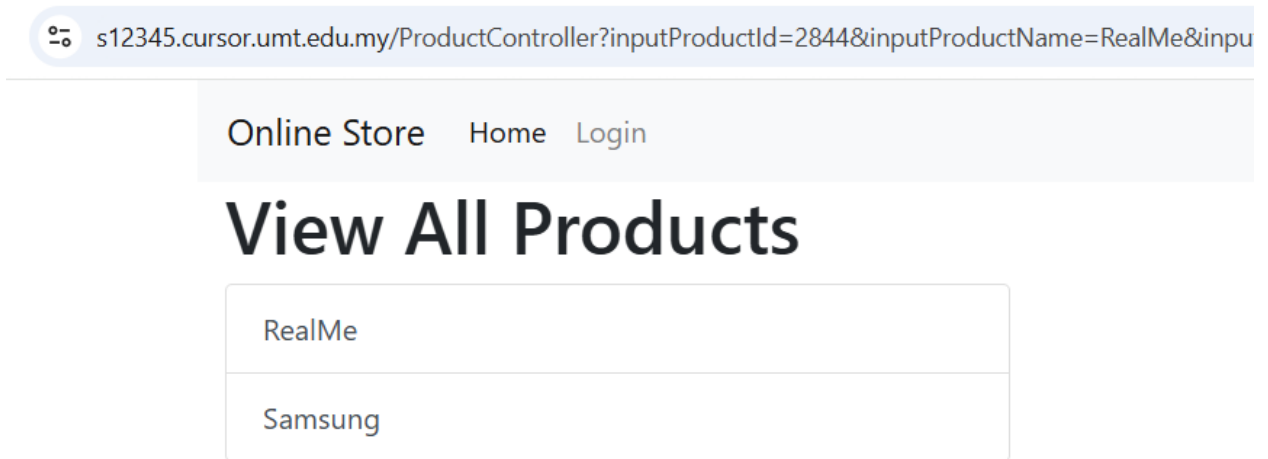


The screenshot shows a web browser at the URL `s12345.cursor.umt.edu.my/ProductController`. The page has a header with links for 'Online Store', 'Home', and 'Login', and a search bar. The main heading is 'View Product [Product ID: 2844]'. Below this, there are four input fields: 'Product Name' with the value 'RealMe', 'Category' with the value 'Mobile Phone', 'Price' with the value '700.0', and 'Quantity' with the value '300'. At the bottom of the form, there are two buttons: 'View All Products' and 'Update'.



The screenshot shows the same web browser at a URL with query parameters: `s12345.cursor.umt.edu.my/ProductController?inputProductId=2844&inputProductName=RealMe&inputCategory=Mobile+Phone&inputProductPrice=700.0&input...`. The page has the same header as the previous screenshot. The main heading is 'Update Product [Product ID: 2844]'. Below this, there are four input fields: 'Product Name' with the value 'RealMe', 'Category' with the value 'Mobile Phone' and a dropdown arrow, 'Price' with the value '700.0', and 'Quantity' with the value '300'. At the bottom of the form, there are three buttons: 'Cancel', 'Delete', and 'Save'.

19. Then finally, view all record. If everything went well, proceed with other modules in your web application.



20. What happen if you see an error? Or nothing showed up when you open the URL of a JSP page? Or it works on localhost but not at a remote server? First of all, make sure the letter you are using is correct. As an example, "OnlineStore" is considered different from "onlinestore", also "book" is different from "Book". If you are using Windows for development, it usually works because Windows does not consider the case of the letters; known as case insensitive. On the other hand, the remote server we are using is a Linux based operating system and it is case sensitive.

TASK 4: UPLOADING FILE TO CURSOR@UMT SERVER

OBJECTIVE

To prepare a servlet and JSP page for file upload.

TASK DESCRIPTION

There will be a requirement where a system needs to provide a file upload feature for the user. Besides using the database to store the files, whether it is an image file, pdf or whatsoever, direct upload to a remote folder is also an option. So, in this task, the student will prepare a servlet and a JSP page for the file upload function. It is important to check the type of file before the file being written to the server. Input validation for file upload is recommended to be done at the client and server-side to prevent cyber attacks.

ESTIMATED TIME

60 Minutes

STEPS:

1. Before we code the JSP page and servlet for handling file upload, let's prepare the location for the uploaded file in the CURSOR@UMT remote server.

2. First of all, log in to CURSOR@UMT user panel. Go to **File Management > File Manager**, then on **Folder** on the menu to create a new folder.

 **Folder**  File  Upload  All ☐ None  Delete  Copy  Move  Compress  Permissi

 /

Name	Size	Date	Over
 .conf		20/06/21 07:12	
 .trash		21/06/21 04:52	
 backupcwp		20/06/21 04:02	
 cwp_stats		20/06/21 07:13	
 public_html		18/06/21 07:11	
 tmp		20/06/21 04:02	
 .bash_logout	18 bytes	27/05/21 03:09	
 .bash_profile	141 bytes	27/05/21 03:09	

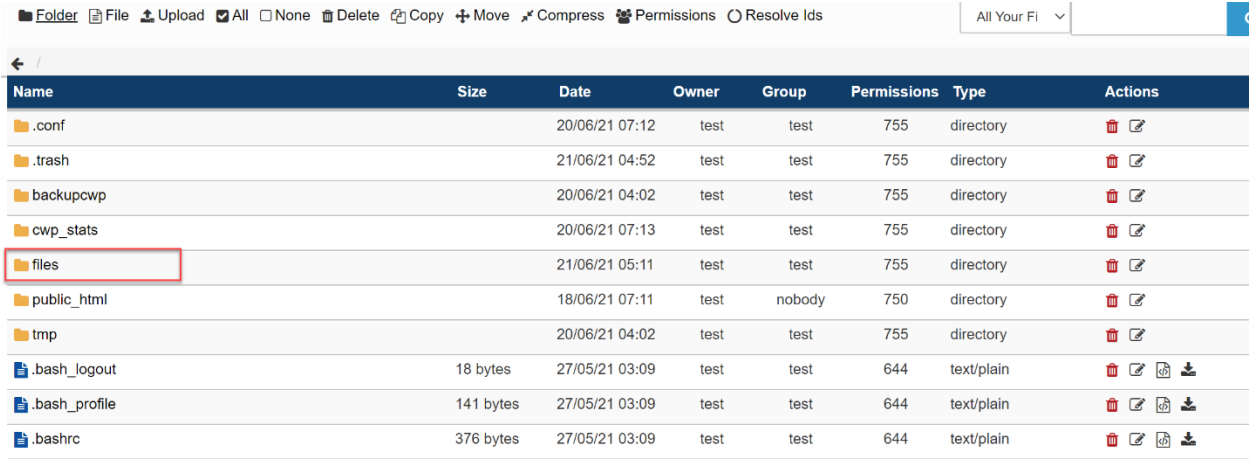
3. Give a suitable name for the folder. In this case, the folder named as **files**. Click **Save** after complete.

New Directory 

Path: /home/test/
New Directory:

Save

- Now, you will see the folder in the list of directory. The full path to the directory will be **/home/test/files**, where **test** is the user name given by the administrator. So, a general form of the folder path could be **/home/[username]/[name of your custom folder]**. Close the window after complete.



Name	Size	Date	Owner	Group	Permissions	Type	Actions
.conf		20/06/21 07:12	test	test	755	directory	
.trash		21/06/21 04:52	test	test	755	directory	
backupcwp		20/06/21 04:02	test	test	755	directory	
cwp_stats		20/06/21 07:13	test	test	755	directory	
files		21/06/21 05:11	test	test	755	directory	
public_html		18/06/21 07:11	test	nobody	750	directory	
tmp		20/06/21 04:02	test	test	755	directory	
.bash_logout	18 bytes	27/05/21 03:09	test	test	644	text/plain	
.bash_profile	141 bytes	27/05/21 03:09	test	test	644	text/plain	
.bashrc	376 bytes	27/05/21 03:09	test	test	644	text/plain	

- It is time to prepare the codes for file upload. Open the Netbeans, create a new project or using the existing project.
- Then, create a new JSP file to prepare for file upload from the user. You can name it with any suitable name. In this example, the file name is **uploadfile.jsp**. Copy the codes in the screenshots to the file.

```

<!--
Document      : uploadfile
Created on    : Jun 17, 2021, 8:17:22 AM
Author       : Fakhru Adli
-->

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Example of File Upload</title>
</head>
<body>
File: <br />
<form action="UploadFileController" method="post"
      enctype="multipart/form-data">
  <input type="file" name="fileToBeUploaded" size="50" />
  <br />
  <input type="submit" value="Upload" />
</form>
</body>
</html>

```

- Next, create a new servlet to handle the file upload. Name the servlet as **UploadFileController** (or any suitable name). Remember to tick at **Add information to deployment descriptor (web.xml)** when required by Netbeans.

8. Before copying the sample codes, include the required libraries for file upload. Those libraries can be found at https://commons.apache.org/proper/commons-fileupload/download_fileupload.cgi and https://commons.apache.org/proper/commons-fileupload/download_fileupload.cgi. Extract the .zip files and import the .jar files to your Netbeans project.
9. Now, copy the following codes to the servlet **UploadFileController** file. Put the codes in **doPost()** method. Change the **/home/test/files** to the valid location as being defined in Step 1 to Step 4 previously.

```
79  @Override
80  protected void doPost(HttpServletRequest request, HttpServletResponse response)
81      throws ServletException, IOException {
82      try {
83          if (ServletFileUpload.isMultipartContent(request)) {
84              try {
85                  List<FileItem> multipart = new ServletFileUpload(new DiskFileItemFactory()).parseRequest(request);
86                  for (FileItem item : multipart) {
87                      if (!item.isFormField()) {
88                          String name = getUniqueFileName();
89                          try {
90                              /* Only write to the disk if file is in the list
91                               isImageFile() list
92                              */
93                              if (isImageFile(name)) {
94                                  item.write(new File("/home/test/files" + File.separator + name));
95                              }
96                          } catch (Exception ex) {
97                              System.out.println("Not an image file");
98                          }
99                      }
100                  }
101              }
102          }
103          //File uploaded successfully
104          request.setAttribute("message", "File Uploaded Successfully");
105          } catch (Exception ex) {
106              request.setAttribute("message", "File Upload Failed due to " + ex);
107          }
108      } else {
109          request.setAttribute("message", "No File found");
110      }
111      request.getRequestDispatcher("/result.jsp").forward(request, response);
112  }
113  catch (IOException | ServletException ex) {
114      Logger.getLogger(ProductController.class
115          .getName()).log(Level.SEVERE, null, ex);
116  }
117  }
118  }
119  }
```

10. Remember to put the following methods to the file as well. The **getUniqueFileName()** method is to generate a unique file name with a specified extension. In this example, the extension is .jpeg. You may change the file type accordingly. The second method is the **isImageFile()** which will check the given file type is in the defined list or not. It is always important to check the file type to prevent a malicious file from being uploaded.

```
public String getUniqueFileName() {
    Random random = new Random();
    String ext = ".jpeg";
    String filename = String.format("%s%s", System.currentTimeMillis(),
        random.nextInt(100000) + ext);
    return filename;
}

public boolean isImageFile(String fileName) {
    return Arrays.asList("png", "gif", "jpg", "jpeg", "tif", "tiff", "bmp")
        .contains(org.apache.commons.io.FilenameUtils.getExtension(fileName));
}
```

11. The second JSP file we need to create is a file to read the message attribute created in Step 9. It is just a simple page that displays the status of the upload process. Name the file as **result.jsp**.

```
<%--
    Document      : result
    Created on    : Jun 17, 2021, 8:36:15 AM
    Author       : Fakhru Adli
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>File Upload</title>
    </head>
    <body>
        <%
            String msg = (String) request.getAttribute("message");
            out.println(msg);
        %>
    </body>
</html>
```

12. Save the files and move to the next task for retrieving the uploaded files.

TASK 5: RETRIEVING FILE FROM CURSOR@UMT SERVER

OBJECTIVE

The objective of this task is to retrieve to file from the server so that it can be shown to the web page or downloaded to the user local machine.

TASK DESCRIPTION

For this task, the student will create a servlet to handle the file retrieval, then a JSP page to show the file.

ESTIMATED TIME

60 Minutes

STEPS:

1. Open a Netbeans if not done before, then create a new servlet. Name the servlet as **filehandler** (or any suitable name) and make sure the configuration is similar to the following screenshot. Click **Finish** when ready.

New Servlet

Steps

1. Choose File Type
2. Name and Location
3. **Configure Servlet Deployment**

Configure Servlet Deployment

Register the Servlet with the application by giving the Servlet an internal name (Servlet Name). Then specify patterns that identify the URLs that invoke the Servlet. Separate multiple patterns with commas.

☒ Add information to deployment descriptor (web.xml)

Class Name: onlinestore.filehandler

Servlet Name: filehandler

URL Pattern(s): /filehandler/*

Initialization Parameters:

Name	Value
------	-------

New Edit... Delete

< Back Next > Finish Cancel Help

2. On the servlet, copy the codes below. Alternatively, you can do the import automatically later. In the code below, we defined the default buffer size which you can change later. The default size now is 10KB. In addition, we also specify the location of the files to be retrieved. Put the path based on your configuration.

```
import java.io.BufferedReader;
import java.io.BufferedOutputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.PrintWriter;
import java.net.URLDecoder;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import static org.apache.commons.io.IOUtils.close;

/**
 *
 * @author Fakhrul Adli
 */
public class filehandler extends HttpServlet {

    private static final int DEFAULT_BUFFER_SIZE = 10240; // 10KB.
    private String filePath;

    public void init() throws ServletException {

        // Define base path somehow. You can define it as init-param of the servlet.
        this.filePath = "/home/test/files";

        // In a Windows environment with the Applicationserver running on the
        // c: volume, the above path is exactly the same as "c:\files".
        // In UNIX, it is just straightforward "/files".
    }
}
```

3. Next, copy the rest of the codes as shown below. Remember to put them in **doGet()** method.

```
75  @Override
76  protected void doGet(HttpServletRequest request, HttpServletResponse response)
77      throws ServletException, IOException {
78      // Get requested file by path info.
79      String requestedFile = request.getPathInfo();
80
81      // Check if file is actually supplied to the request URI.
82      if (requestedFile == null) {
83          // Do your thing if the file is not supplied to the request URI.
84          // Throw an exception, or send 404, or show default/warning page, or just ignore it.
85          response.sendError(HttpServletResponse.SC_NOT_FOUND); // 404.
86          return;
87      }
88
89      // Decode the file name (might contain spaces and on) and prepare file object.
90      File file = new File(filePath, URLDecoder.decode(requestedFile, "UTF-8"));
91
92      // Check if file actually exists in filesystem.
93      if (!file.exists()) {
94          // Do your thing if the file appears to be non-existing.
95          // Throw an exception, or send 404, or show default/warning page, or just ignore it.
96          response.sendError(HttpServletResponse.SC_NOT_FOUND); // 404.
97          return;
98      }
99
100     // Get content type by filename.
101     String contentType = getServletContext().getMimeType(file.getName());
102
103     // If content type is unknown, then set the default value.
104     // For all content types, see: http://www.w3schools.com/media/media\_mimeref.asp
105     // To add new content types, add new mime-mapping entry in web.xml.
106     if (contentType == null) {
107         contentType = "application/octet-stream";
108     }
109
110     // Init servlet response.
111     response.reset();
112     response.setBufferSize(DEFAULT_BUFFER_SIZE);
113     response.setContentType(contentType);
114     response.setHeader("Content-Length", String.valueOf(file.length()));
115     response.setHeader("Content-Disposition", "attachment; filename=\"" + file.getName() + "\"");
116
117     // Prepare streams.
118     BufferedInputStream input = null;
119     BufferedOutputStream output = null;
120
121     try {
122         // Open streams.
123         input = new BufferedInputStream(new FileInputStream(file), DEFAULT_BUFFER_SIZE);
124         output = new BufferedOutputStream(response.getOutputStream(), DEFAULT_BUFFER_SIZE);
125
126         // Write file contents to response.
127         byte[] buffer = new byte[DEFAULT_BUFFER_SIZE];
128         int length;
129         while ((length = input.read(buffer)) > 0) {
130             output.write(buffer, 0, length);
131         }
132     } finally {
133         // Gently close streams.
134         close(output);
135         close(input);
136     }
137 }
```

- Now, let's create the page where the content of the file will be displayed or downloaded. Create a new JSP file and copy the following codes. This file is retrieving the image file with a random file name and show it to the user through the browser. The file name is hardcoded. However, in a real implementation, you could retrieve the file name from a table in the database.

```
1  <%--
2      Document    : viewImage
3      Created on  : Jun 20, 2021, 12:34:27 PM
4      Author     : Fakhru Adli
5  --%>
6
7  <%@page contentType="text/html" pageEncoding="UTF-8"%>
8  <!DOCTYPE html>
9  <html>
10     <head>
11         <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
12         <title>View Image</title>
13     </head>
14     <body>
15         <h1>View Uploaded Image</h1>
16         <!-- This is just an example. In a real implementation, the file name
17              could be retrieved from the database -->
18         
19     </body>
20 </html>
21
```

- After done with all the coding process above, let's save all the files. Then do the **Clean & Build**.
- Finally, upload the WAR file as being described in Task 3. Well done!

FAQ PENGGUNAAN SERVER CURSOR_ (BAHASA MALAYSIA)

1. Apakah itu CURSOR_?

CURSOR_ ialah platform Java Web Hosting yang disediakan untuk membantu pelajar FSKM Computer Science menerbitkan aplikasi web mereka, khususnya aplikasi berasaskan JSP/Servlet, WAR file, dan MySQL database. Pelajar boleh menggunakan CURSOR_ untuk menguji aplikasi dalam persekitaran sebenar seperti server Linux, bukan hanya di localhost.

2. Mengapa aplikasi saya berfungsi di localhost tetapi tidak berfungsi di CURSOR_?

Antara punca biasa ialah:

- Nama fail, folder, class, servlet, atau URL tidak sama dari segi huruf besar dan kecil.
- Database name, username, atau password tidak betul.
- WAR file tidak dinamakan mengikut nombor matrik.
- SQL file belum diimport ke phpMyAdmin.
- Link dalam JSP ditulis secara hardcoded dan tidak menggunakan `{pageContext.request.contextPath}`.
- Servlet mapping dalam web.xml tidak sepadan dengan link dalam JSP.
- Library seperti MySQL Connector/J atau JSTL tidak dimasukkan dengan betul.

3. Mengapa pelajar tidak boleh menggunakan Jakarta Servlet/Jakarta EE dalam projek CURSOR_?

Untuk server CURSOR_ semasa, pelajar perlu menggunakan Java EE/Servlet berasaskan pakej javax, bukan Jakarta EE yang menggunakan pakej jakarta. Ini kerana aplikasi pelajar akan dideploy sebagai WAR ke Apache Tomcat versi yang digunakan oleh CURSOR_. Tomcat yang disediakan oleh CURSOR_ menyokong Java Servlet API berasaskan javax.servlet, manakala pakej jakarta.servlet biasanya digunakan untuk Tomcat versi yang berlainan.

Kesilapan biasa berlaku apabila pelajar menggunakan tutorial atau library baharu yang menghasilkan import seperti berikut:

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

Kod seperti ini mungkin berjaya dikompil jika dependency Jakarta dimasukkan, tetapi berisiko gagal apabila dideploy ke server CURSOR_ yang menggunakan Tomcat versi tertentu. Gunakan import berikut:

```
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
```

Bagi projek CURSOR_, elakkan juga penggunaan @WebServlet annotation jika manual atau projek kursus menetapkan semua servlet mapping dibuat dalam web.xml. Pastikan mapping servlet diletakkan dengan jelas dalam web.xml seperti contoh berikut:

```
<servlet>
  <servlet-name>LoginServlet</servlet-name>
  <servlet-class>controller.LoginServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>LoginServlet</servlet-name>
  <url-pattern>/LoginServlet</url-pattern>
</servlet-mapping>
```

Ringkasan peraturan untuk pelajar:

- Gunakan javax.servlet.*, bukan jakarta.servlet.*.
- Gunakan Tomcat compatible dependencies yang serasi dengan CURSOR_.
- Jangan campur javax dan jakarta dalam projek yang sama.
- Untuk JSP/JSTL lama, gunakan dependency atau JAR yang sesuai dengan javax, bukan Jakarta JSTL.
- Jika menggunakan Maven, servlet API perlu berada dalam scope provided kerana Tomcat sudah menyediakan Servlet API semasa runtime.

Contoh dependency Maven yang lebih sesuai untuk projek CURSOR_ ialah:

```
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>4.0.1</version>
  <scope>provided</scope>
</dependency>
```

Jika pelajar mendapat error seperti ClassNotFoundException, NoClassDefFoundError, atau servlet tidak dapat dipanggil selepas deploy, semak dahulu sama ada projek tersalah menggunakan pakej jakarta.servlet. Tukar semula kepada javax.servlet dan rebuild WAR sebelum upload semula ke CURSOR_.

4. **Mengapa saya perlu menggunakan `${pageContext.request.contextPath}` dalam JSP?**

`${pageContext.request.contextPath}` digunakan untuk mendapatkan root path sebenar aplikasi web secara automatik. Ini penting kerana aplikasi mungkin berjalan dengan path berbeza antara localhost dan CURSOR_.

```
http://localhost:8080/OnlineStore/  
https://s12345.cursor.umt.edu.my/
```

5. **Di mana saya patut gunakan `${pageContext.request.contextPath}`?**

Gunakan pada semua path yang merujuk kepada fail atau servlet dalam aplikasi.

```
<a href="${pageContext.request.contextPath}/dashboard.jsp">Dashboard</a>  
<form action="${pageContext.request.contextPath}/AddProductServlet" method="post">  
<link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">  
<script src="${pageContext.request.contextPath}/js/app.js"></script>  

```

6. **Apakah kesilapan biasa jika tidak menggunakan context path?**

Jika pelajar menulis path secara relatif atau hardcoded, link mungkin berfungsi di satu halaman tetapi gagal apabila JSP berada dalam subfolder atau selepas deploy ke server.

```
<!-- Kurang sesuai -->  
<form action="LoginServlet" method="post">  
<link rel="stylesheet" href="css/style.css">  
  
<!-- Lebih selamat -->  
<form action="${pageContext.request.contextPath}/LoginServlet" method="post">  
<link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
```

7. **Apakah nama WAR file yang betul untuk upload ke CURSOR_?**

WAR file mesti dinamakan menggunakan nombor matrik pelajar dan mengekalkan extension .war.

```
s12345.war
```

8. Di mana lokasi WAR file selepas build?

Jika menggunakan NetBeans Ant project, WAR file biasanya berada di folder `dist`. Jika menggunakan Maven project, WAR file biasanya berada di folder `target`.

```
[Project Location]/[Project Name]/dist/  
[Project Location]/[Project Name]/target/
```

9. Mengapa saya perlu upload WAR file di folder / dalam File Manager?

Pelajar perlu pastikan mereka berada di folder root user panel, iaitu `/`, sebelum upload WAR file. Jika WAR file diletakkan di folder lain, aplikasi mungkin tidak dikesan atau tidak dideploy dengan betul.

10. Apakah format database name yang betul?

Database name perlu bermula dengan username pelajar, diikuti underscore `_`. Contohnya, jika username ialah `s12345` dan nama database asal ialah `onlinestore`, nama database di `CURSОР_` ialah `s12345_onlinestore`.

```
s12345_onlinestore
```

11. Bolehkah saya guna nama database asal seperti `onlinestore` sahaja?

Tidak boleh dan tidak akan berjaya di server `CURSОР_`. Di server hosting, database biasanya perlu menggunakan prefix username.

```
[username]_[databasename]  
s12345_onlinestore
```

12. Adakah huruf besar dan kecil penting?

Ya, sangat penting. Server `CURSОР_` menggunakan Linux yang bersifat case-sensitive. Oleh itu, nama berikut dianggap berbeza:

```
s12345 ≠ S12345  
onlinestore ≠ OnlineStore  
book ≠ Book  
login.jsp ≠ Login.jsp
```

13. Apakah konfigurasi database yang perlu diubah sebelum build WAR?

Pelajar perlu menyemak tiga perkara utama: database name, username, dan password. Akaun root biasanya hanya untuk localhost dan tidak boleh digunakan di CURSOR_.

Perkara	Contoh Localhost	Contoh CURSOR_
Database name	onlinestore	s12345_onlinestore
Username	root	s12345
Password	password local	password diberi admin

```
String url = "jdbc:mysql://localhost:3306/s12345_onlinestore";  
String username = "s12345";  
String password = "password_yang_diberi";
```

14. Adakah saya perlu export database dari localhost?

Ya. Pelajar perlu export database dari phpMyAdmin di localhost untuk mendapatkan fail .sql. Fail SQL ini kemudian perlu diimport semula ke phpMyAdmin di CURSOR_. Dua fail penting perlu disediakan sebelum upload ke server ialah WAR file dan SQL file.

15. Selepas import SQL, mengapa table saya masih tidak wujud?

Punca biasa ialah pelajar import SQL ke database yang salah, database belum dipilih sebelum import, SQL file tidak lengkap, export dari localhost gagal, nama database dalam code tidak sama, atau table name tidak sama dari segi huruf besar/kecil.

16. Mengapa saya mendapat error "Access denied for user"?

Error ini biasanya bermaksud username atau password database salah. Pastikan username bukan lagi root dan password adalah password yang diberikan oleh admin CURSOR_.

```
String username = "s12345";  
String password = "password_yang_diberi";
```

17. Mengapa saya mendapat error “Unknown database”?

Error ini biasanya bermaksud nama database dalam code tidak sama dengan database yang dicipta di phpMyAdmin.

```
// Salah
jdbc:mysql://localhost:3306/onlinestore

// Betul
jdbc:mysql://localhost:3306/s12345_onlinestore
```

18. Mengapa saya mendapat error “ClassNotFoundException: com.mysql.cj.jdbc.Driver”?

Ini bermaksud MySQL JDBC driver tidak dimasukkan dengan betul dalam projek. Untuk Maven, pastikan dependency MySQL Connector/J dimasukkan dalam pom.xml. Untuk NetBeans Ant project, pastikan .jar MySQL connector dimasukkan dalam Libraries project.

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.4.0</version>
</dependency>
```

19. Apakah URL untuk menguji aplikasi selepas deploy?

Gunakan format berikut:

```
https://[nombor_matrik].cursor.umat.edu.my
https://s12345.cursor.umat.edu.my
```

20. Selepas deploy, apakah fungsi yang perlu saya uji?

Pelajar perlu menguji semua fungsi utama, khususnya CRUD.

Fungsi	Maksud
Create	Tambah rekod
Read	Papar rekod
Update	Kemaskini rekod
Delete	Padam rekod

21. Mengapa halaman utama keluar tetapi fungsi tambah/kemaskini/padam tidak berfungsi?

Punca biasa ialah form action salah, servlet mapping tidak betul, tidak menggunakan context path, nama servlet dalam form tidak sama dengan web.xml, database connection gagal, atau SQL query masih merujuk kepada database/table local.

```
<form action="${pageContext.request.contextPath}/AddProductServlet" method="post">

<servlet-mapping>
    <servlet-name>AddProductServlet</servlet-name>
    <url-pattern>/AddProductServlet</url-pattern>
</servlet-mapping>
```

22. Perlukah saya guna PreparedStatement?

Ya. Pelajar perlu menggunakan PreparedStatement untuk query yang menerima input pengguna bagi mengurangkan risiko SQL Injection.

```
// Lebih selamat
String sql = "SELECT * FROM users WHERE email = ? AND password = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, email);
ps.setString(2, password);

// Elakkan
String sql = "SELECT * FROM users WHERE email='" + email + "' AND password='" + password + "'";
```

23. Apakah kesilapan biasa semasa upload file dalam aplikasi?

Kesilapan biasa ialah folder upload belum dibuat, path folder dalam servlet masih menggunakan contoh lama, username dalam path tidak ditukar, file type tidak divalidasi, permission folder tidak sesuai, atau nama fail mengandungi aksara pelik.

```
// Betul, ikut username sendiri
String uploadPath = "/home/s12345/files";

// Salah jika masih guna contoh lama
String uploadPath = "/home/test/files";
```

24. Mengapa validasi file upload penting?

Validasi file upload penting untuk mengelakkan pengguna memuat naik fail berbahaya seperti script, executable file, atau fail yang menyamar sebagai image. Validasi sebaiknya dilakukan di client-side dan server-side.

```
private boolean isImageFile(String fileName) {
    String lower = fileName.toLowerCase();
    return lower.endsWith(".jpg") ||
           lower.endsWith(".jpeg") ||
           lower.endsWith(".png");
}
```

25. Mengapa gambar/CSS/JS tidak keluar selepas deploy?

Punca biasa ialah path yang salah. Gunakan context path supaya fail static seperti image, CSS dan JavaScript dapat dicapai dengan betul.

```

<link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
<script src="${pageContext.request.contextPath}/js/script.js"></script>
```

26. Apakah contoh standard untuk form action yang betul?

Gunakan context path dalam form action supaya servlet dapat dicapai dengan betul selepas deployment.

```
<form action="${pageContext.request.contextPath}/LoginServlet" method="post">
  <input type="text" name="email">
  <input type="password" name="password">
  <button type="submit">Login</button>
</form>
```

27. Apakah contoh standard untuk link menu yang betul?

Contoh menu yang lebih selamat adalah seperti berikut:

```
<a href="${pageContext.request.contextPath}/index.jsp">Home</a>
<a href="${pageContext.request.contextPath}/products.jsp">Products</a>
<a href="${pageContext.request.contextPath}/cart.jsp">Cart</a>
<a href="${pageContext.request.contextPath}/LogoutServlet">Logout</a>
```

28. Apakah contoh standard untuk redirect dalam Servlet?

Dalam servlet, jangan hardcode nama project localhost. Gunakan request.getContextPath().

```
// Betul
response.sendRedirect(request.getContextPath() + "/dashboard.jsp");

// Elakkan
response.sendRedirect("/OnlineStore/dashboard.jsp");
response.sendRedirect("http://localhost:8080/OnlineStore/dashboard.jsp");
```

29. Apakah contoh standard untuk forward dalam Servlet?

Untuk forward, biasanya tidak perlu letak `request.getContextPath()` kerana forward berlaku di server-side.

```
request.getRequestDispatcher("/dashboard.jsp").forward(request, response);
```

Situasi	Guna context path?
Link HTML/JSP	Ya
Form action	Ya
CSS/JS/Image path	Ya
<code>response.sendRedirect()</code>	Ya, guna <code>request.getContextPath()</code>
<code>request.getRequestDispatcher()</code>	Tidak perlu

30. Checklist sebelum upload WAR ke CURSOR_

Semakan	Status
Semua query penting menggunakan PreparedStatement	☐
Database name sudah ditukar kepada format [username]_[database]	☐
Username database bukan lagi root	☐
Password database betul	☐
Semua link/form action guna <code>\${pageContext.request.contextPath}</code>	☐
Nama fail/folder/class disemak huruf besar/kecil	☐

Project berjaya Clean & Build	☐
WAR file dijumpai di dist atau target	☐
WAR file dinamakan sebagai nombor matrik, contoh s12345.war	☐
SQL file sudah diexport dari localhost	☐
WAR file diupload ke folder / dalam File Manager	☐
Database sudah dicipta dalam phpMyAdmin CURSOR_	☐
SQL file sudah diimport	☐
Aplikasi diuji melalui https://[matric].cursor.umt.edu.my	☐
CRUD diuji satu per satu	☐

31. Apakah maklumat yang perlu disediakan sebelum meminta bantuan pensyarah/admin?

Sebelum meminta bantuan, pelajar perlu sediakan maklumat berikut supaya masalah dapat dikenal pasti dengan lebih cepat:

- Screenshot error
- URL aplikasi
- Nama WAR file yang diupload
- Nama database dalam phpMyAdmin
- Bahagian code database connection
- Bahagian form action atau link yang bermasalah
- Servlet mapping dalam web.xml
- Nyatakan sama ada konfigurasi database dibuat di dalam satu fail sahaja atau disalin di setiap fail servlet atau JSP.